

barehands — anatomy edition

Hold a 3D heart in your bare hands, cut it open, point at a structure and hear what it is, then work a sourced pathology case. All you need is a webcam and Chrome.

This is a fork of [jaredrhod/barehands](https://github.com/jaredrhod/barehands) (<https://github.com/jaredrhod/barehands>) by Jared Rhodenizer, the hand-tracked glass interface. Upstream provides the body: hand tracking, glass cards, 3D props, gestures. This fork adds an anatomy and pathology teaching layer on top and keeps everything else merge-clean, so upstream gesture fixes keep flowing in.

Education and training only. This is not a medical device. It doesn't diagnose real patients, and nothing it shows is clinical judgment about a real person. Every structure name, teaching line and pathology claim comes from content that was written with citations and independently reviewed; nothing is made up at runtime.

What you can do

Stage an organ	A 16-part anatomical heart (chambers, great vessels, coronaries, all four valves), each part a separately named mesh.
Explode it	An empty pinch dragged sideways pulls the parts apart; the camera backs off so nothing leaves the frame.
Cut it open	Press X to arm, then swipe a flat palm across the organ: your palm's plane becomes the incision. Spread two fingers to scrub the cut deeper. Cut faces render solid. Anatomical presets are available too: sagittal, coronal, axial.
Point and ask	Rest a hand on a structure. The board publishes what you're pointing at, and <code>bin/narrate.sh</code> says its name aloud (add <code>--teach</code> to hear what it does).
Isolate, hide, label, x-ray, peel	Seven validated board verbs your AI can call.
Take a case	The examiner skill runs a graded session: read the vignette, explore the lesion with your hands, commit to a diagnosis, get graded against a rubric, then a debrief.

Run it

```
python3 server.py
```

(python server.py on Windows.) Open **http://127.0.0.1:8794/stage.html** (**http://127.0.0.1:8794/stage.html**) in **Chrome**, allow the camera, and wave. The server uses only the Python standard library. On first load the page pulls MediaPipe and three.js from CDNs, then caches them.

Your config is `barehands.json`, and it's untracked: copy `barehands.json.example`. The first time, you can open the repo in Claude Code and say *“read barehands.md and set me up”* to have the setup done for you.

Then tap the ring → tap the **Props** orb → open `models/heart/heart.glb`.

Tracker keys

key	does
X	arm / disarm the hand-driven cut (auto-disarms after 3 min idle)
R	re-read note folders
C	cycle cameras
D	debug overlay
P / G	pose sampler / cut-gesture sampler (for tuning)

While a cut is armed, that hand can’t grab, throw or explode-scrub the organ, and the clap won’t clear the board. This is intentional: two flat hands is the incision pose. Press **X** again to get normal gestures back.

Drive it with an AI

Any program that can curl localhost can drive the board, no AI required. Two scripts give it hands and eyes on the board:

```
bin/board.sh '{"a":"label","structures":["left_ventricle","mitral_valve"]}'
bin/board.sh '{"a":"cut","plane":"coronal","t":0.5}'
bin/board.sh '{"a":"isolate","structure":"left_coronary_artery"}'
bin/board.sh '{"a":"xray","focus":"aortic_valve"}'
bin/board-state.sh          # what's on the board, incl. "POINTING AT:"
```

verb	does
cut	preset or custom plane, depth t , arm , off
isolate / hide	show only one structure / hide one
label	up to 12 structures, or "all"
xray	translucent view, with an optional opaque focus
layer	peel down to a dissection depth
lesion	render a case's lesion on the model

The server validates every argument (enums, ranges, id patterns) and rejects anything malformed with a 400. The full contract is in `schemas/verbs.md` (`schemas/verbs.md`).

Agent skills live in `skills/` . Load the matching one before doing the task:

skill	for
board-driver (skills/board-driver/SKILL.md)	driving the board and the “what is this?” identification loop
examiner (skills/examiner/SKILL.md)	running a graded case session
pathology-authoring (skills/pathology-authoring/SKILL.md)	writing a cited pathology teaching doc
case-authoring (skills/case-authoring/SKILL.md)	turning a verified pathology doc into a playable case
gesture-tuning (skills/gesture-tuning/SKILL.md)	fitting the cut-gesture thresholds to a real hand

Helper scripts: `bin/narrate.sh` [`--teach`] [`--dwell N`] [`--voice V`] (macOS `say`) and `bin/point-watch.sh` , which watches the pointing loop live and tells a tracking failure (`hands 0`) apart from a pointing failure (`pt none`).

What’s in the box

organ	structures	cases
Heart (BodyParts3D, CC-BY 4.0)	16	heart-anterior-stemi-01 : anterior MI at 48 h, verified

Adding an organ means three things: a GLB with named meshes, an `anatomy.json` sidecar that maps meshes to structures (with teaching text and sources), and a `LICENSE.md` that credits the source. Adding a case means a cited pathology doc, a case file, and independent review. `schemas/validate.py` checks both:

```
python3 schemas/validate.py media/models/heart/anatomy.json media/models/heart/heart.glb
python3 schemas/validate.py --case cases/heart-anterior-stemi-01.json media/models/heart/anatomy.json
```

Layout

<code>js/medical/</code>	<code>core.js</code> (sidecars, verbs, pointing) · <code>render.js</code> (cuts, caps, x-ray, labels) · <code>gesture.js</code> (hand-driven cut)
<code>schemas/</code>	<code>anatomy</code> + case schemas, verb contract, <code>validate.py</code>
<code>media/models/</code>	organs + <code>anatomy.json</code> sidecars + <code>LICENSE.md</code> (media/holo/: hologram copies)
<code>pathology/</code>	cited teaching docs – the source of truth for each disease
<code>cases/</code>	playable, graded cases
<code>review/</code>	independent review artifacts for each case
<code>skills/</code>	agent procedures (see above)
<code>tests/</code>	contract, gesture, and visual acceptance suites
<code>KNOWLEDGE.md</code>	the knowledge ledger: laws, verified facts, lessons, open items

Everything else (`stage.html`, `server.py`, `barehands.md`, `sample-notes/`) is upstream barehands. `stage.html` changes only through single, marked `MEDICAL HOOK` lines, and every hook is an optional call, so a broken medical module falls back to plain barehands instead of freezing tracking.

Tests

The contract and visual suites need the server running; the gesture suite doesn't:

```
python3 tests/test_contract.py    # 77 assertions, ~5 s – verbs, airlock, answer isolation, sidecar
node    tests/test_gesture.mjs    # 54 assertions – cut-gesture math and guards
python3 tests/test_visual.py      # 17 renders → tests/_shots/heart-visual-matrix.png, ~5 min
```

Close the barehands tab before running the board-state checks. An open tracker owns the state bus, so those checks report SKIPPED rather than fail. Pointing accuracy and gesture thresholds can only be verified by a person at the webcam. See `tests/README.md` (`tests/README.md`).

Knowledge ledger

`KNOWLEDGE.md` (`KNOWLEDGE.md`) records what this fork has learned and why: the rules that can't be broken, measured facts about the assets and runtime, every field bug with its real cause, the review status of each piece of content, and what's still unverified. Read it before changing the medical layer, and add to it when you learn something.

Safety rules

- **The media airlock never widens.** The board stages files only from `media/`.
- **The `/cmd` allowlist grows only through validated verbs**, never a passthrough.
- **Answer isolation.** `/case-render` never exposes the vignette, diagnosis or teaching, and the examiner never labels the structure a lesion sits on.
- **Unverified cases don't run.** The examiner refuses any case whose `review.status` isn't `verified`.
- **Everything is local.** Nothing leaves the machine.

Updating from upstream

A fork can't just pull. Upstream's updater fast-forwards `main`, which either fails on this fork's history or, if forced, buries the medical layer. So `./update.sh` (`update.bat` on Windows) **never merges**. It fetches upstream, shows what's new, and lists the files that upstream *and* this fork have both changed. You then merge deliberately:

```
./update.sh
git merge upstream/main
```

Conflicts can only happen in the handful of upstream files this fork has edited: mainly `stage.html`, `server.py` and `README.md`, plus `bin/board-state.sh`, the updaters, `.gitignore` and `CLAUDE.md`. If a `MEDICAL HOOK` line conflicts, keep both: upstream's new line plus the hook. If `README.md` conflicts, keep this fork's version. Your config, notes and media are untracked and never touched.

For everything upstream (the full gesture set, notes orbs, wiring the ring to your assistant, OBS/streaming mode, gesture troubleshooting), see upstream barehands (<https://github.com/jaredrhod/barehands>), `barehands.md` (`barehands.md`) and `TROUBLESHOOTING.md` (`TROUBLESHOOTING.md`).

Credits

- **barehands** by Jared Rhodenizer (<https://jaredrhod.com>) (@jaredrhod): the hand-tracked interface this fork is built on.
- **Heart model:** BodyParts3D (<https://lifesciencedb.jp/bp3d/>), © The Database Center for Life Science (DBCLS), CC-BY 4.0. Modifications are listed in `media/models/heart/LICENSE.md`.
- Hand tracking: Google MediaPipe (<https://developers.google.com/mediapipe>) (Apache 2.0). 3D: three.js (<https://threejs.org>) (MIT). Both load from public CDNs and aren't redistributed here.
- Pathology sources are cited inline in each `pathology/` doc.

License

AGPL-3.0-or-later, as upstream requires. Upstream code is Copyright (c) 2026 Jared Rhodenizer; this fork's medical additions are released under the same license, with source published as the AGPL requires. If you distribute this or run a modified version as a service, your version must ship under the same license with its source available. Anatomical models are separate data works under their own licenses. Each model folder in `media/models/` has a `LICENSE.md`. Full terms are in `LICENSE (LICENSE)`. For closed-source commercial licensing of upstream barehands, email license@jaredrhod.com (<mailto:license@jaredrhod.com>).